

## Node.js

### Parte I

#### Instalación

Instalación simple, descargar el archivo .msi para Windows y seguir las instrucciones del Wizard. En la instalación personalizada, seleccionar versión LTM para el sistema operativo correspondiente.

#### Práctica

1. **Dados:** Crear el archivo app\_dado.js, en cada interacción se debe simular el lanzado de dos dados, si en 5 intentos hay coincidencia en los resultados obtenidos, el jugador gana (dos dados con el 3) si no, pierde.
2. **Contar palabras:** La aplicación debe contar en un texto la cantidad de apariciones de una determinada variable. El texto debe estar en un archivo formato .txt. La palabra para buscar simplemente se define en una constante.
3. **Generar alumnos.json:** A partir de la tabla alumnos, generar un archivo alumnos.json con los siguientes campos: nro\_alumno, apellido, nombre, dni, nro\_legajo, fecha\_nacimiento, sexo, teléfono, dirección, cod\_postal, estado. El archivo exportado no debe tener espacios a la derecha de los campos tipo char.
4. **Validar Mail y Telefono:** Usando expresiones regulares reporte todos los alumnos que tiene el mail invalido o el teléfono con formato WS. El reporte de todas las incorrecciones encontradas se deben guardar en un archivo "resultado\_test" + milisegundos + ".txt". En donde debe constar apellido y nombre + inconsistencias.
5. **Datos Sistema:** Obtener datos del sistema operativo, cpu, memoria, tiempo encendido, etc. Ver extensiones, instalar ms de vercell.
6. **Server Node:** Crear el programa app\_server.js solo con herramientas propias de node, es decir sin instalar Extensiones. Prever puertos variables. Responder en la raíz / un mensaje en html que el servicio está funcionando.
7. **Recuperar Alumnos:** Programar la ruta /alumnos para que devuelva un json con los datos de alumnos que se obtuvieron de la exportación a un archivo .json desde la tabla alumnos.

8. **Recuperar un Alumno:** Recuperar con Query Params un alumno por nro\_alumno. localhost/alumno?nro=XXXX
9. **Express:** Instalar la extensión express. Programar la respuesta del servidor en funcionamiento.
10. **Recuperar Alumnos:** Devuelva en la ruta /alumnos el json correspondiente usando Express.
11. **Recuperar un Alumno:** Recupere un Alumno usando la ruta alumno/XX con request.params.
12. **Alta Alumno:** Cree un nuevo alumno usando el método randomUUID(), luego realice la misma tarea encontrando el mayor nro\_alumno en alumnos e incrementarlo en 1.

**Instalar REST Client:** Para realizar pruebas usando el archivo test.http, Recordar ingresar en cada prueba ### y la acción.

13. **Modificar Alumno:** Modificar datos de un Alumno exceptuando nro\_alumno, estado y fecha\_nacimiento.
14. **Middleware para permisos:** Mediante middleware debe validar que todos los requerimientos a la API sean de usuario logeados. Para eliminar un Usuario además de logeado debe ser un Usuario Administrador.
15. **Eliminar un Alumno:** Verificar que el estado del alumno a borrar sea Baja(BAJ) y el Usuario sea Administrador.
16. **QueryParams:** Utilice los queryparams para filtrar Alumnos por apellido, estado y sexo.

Ver diferencia entre request.params y request.query.

17. **Paginación:** Implemente paginación de resultados para la consulta anterior de Alumnos.